CASE STUDY 9

MULTI-CURRENCY TRAVEL BUDGET ANALYSIS

Python Programming – Activity-Based Learning

1. Problem Statement

Managing international travel spending is often challenging due to volatile exchange rates and hidden transaction processing fees. Travelers need a system to consolidate multi-currency expenses into a base currency (USD), calculate a standardized cross-border surcharge on each transaction, evaluate total aggregated costs against a fixed budget ceiling, and generate an itemized cost category dashboard.

2. Objectives

- To master automated stream data aggregation using nested structural maps and array loops.
- To normalize volatile foreign currency indices against fixed reference exchange vectors.
- To program operational budget deficit traps and flag severe fiscal boundary violations.
- To display formatted accounting logs utilizing aligned right-justified terminal dashboards.

3. Assumptions

- Base travel operational budget cap is strictly defined at **$5,000.00 USD**.
- Foreign Exchange (FX) base reference rates: 1 EUR = 1.10 USD, 1 JPY = 0.0068 USD, 1 GBP = 1.28 USD.
- Cross-border standard foreign card transaction processing surcharge is set at a flat **2.0%**.
- Expense tracking items are distributed across three distinct strategic operational categories: Accommodation, Transport, and Food.

4. Algorithm

1. Start.
2. Initialize the budget limit to 5000.00 USD and total converted expenditure to 0.0.
3. Define the exchange rate index mappings and cross-border processing rate factor (0.02).
4. Iterate through each raw multi-currency item tuple within the source sequence array.
5. Convert the foreign expense cost to the USD base level by multiplying by its respective exchange scale vector.
6. Compute the transaction fee by taking 2% of the converted amount and add it to the subtotal.
7. Accumulate the loaded subtotal cost directly to the active absolute expenditure tracker variable.
8. Verify whether the total compiled value remains less than or equal to the designated budget cap ceiling.

9. If within bounds, generate a successful spending summary panel; else, trigger a budget threshold overrun warning.
10. Stop.

5. Flowchart

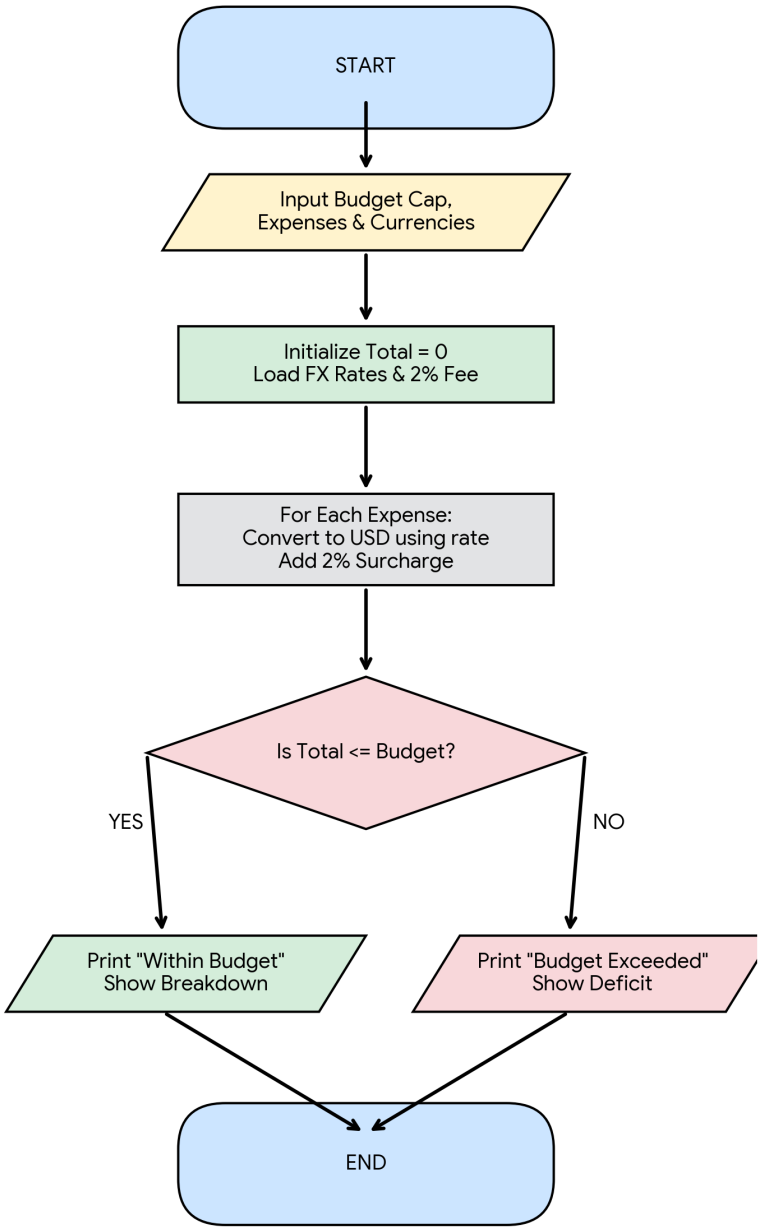


Figure 1: Vertical Logical Flowchart using the Academic Student Method

6. Pseudocode

```
START
    SET budget_cap = 5000.00
    SET total_usd = 0.0
```

```
    SET fx_rates = {"EUR": 1.10, "JPY": 0.0068, "GBP": 1.28, "USD": 1.00}
    SET fee_rate = 0.02

    INPUT list of expenses (amount, currency, category)

    FOR EACH expense IN expenses:
        SET base_rate = fx_rates[expense.currency]
        SET converted_usd = expense.amount * base_rate
        SET transaction_fee = converted_usd * fee_rate
        SET total_item_cost = converted_usd + transaction_fee
        total_usd = total_usd + total_item_cost
    END FOR

    PRINT "Total Consolidated Expenses: ", total_usd

    IF total_usd <= budget_cap THEN
        PRINT "Status: Within Budget Constraints"
    ELSE
        SET deficit = total_usd - budget_cap
        PRINT "Status: Budget Deficit - Over Limit By: ", deficit
    END IF
STOP
```

7. Python Source Code

```
def analyze_travel_budget():
    budget_cap = 5000.00
    total_usd = 0.0
    fee_rate = 0.02

    # Static exchange rates to USD
    fx_rates = {"EUR": 1.10, "JPY": 0.0068, "GBP": 1.28, "USD": 1.00}

    # Mock transactional dataset representing realistic student travel items
    expenses = [
        {"item": "Hotel Paris", "amount": 1200, "currency": "EUR", "cat":
"Accommodation"},
        {"item": "Bullet Train Tokyo", "amount": 45000, "currency": "JPY",
"cat": "Transport"},
        {"item": "London Dinner", "amount": 150, "currency": "GBP", "cat":
"Food"},
        {"item": "Flight Transfer", "amount": 800, "currency": "USD", "cat":
"Transport"}
    ]

    print("=" * 60)
    print(f"{'ITEM':<20} {'AMOUNT':<10} {'CURR':<6} {'CONVERTED (USD)':<16}")
    print("=" * 60)
```

```
    for exp in expenses:
        rate = fx_rates.get(exp["currency"], 1.0)
        conv_usd = exp["amount"] * rate
        surcharge = conv_usd * fee_rate
        final_item_cost = conv_usd + surcharge
        total_usd += final_item_cost
        print(f"{exp['item']:<20} {exp['amount']:<10} {exp['currency']:<6} $
{final_item_cost:>14.2f}")

    print("=" * 60)
    print(f"Total Consolidated Expenses: ${total_usd:.2f}")
    print(f"Allocated Budget Ceiling:   ${budget_cap:.2f}")
    print("-" * 60)

    if total_usd <= budget_cap:
        print("SYSTEM STATUS: SUCCESS [Within Budget Constraints]")
    else:
        deficit = total_usd - budget_cap
        print(f"SYSTEM STATUS: WARNING [Budget Ceilings Violated by ${deficit:.
2f}]")
    print("=" * 60)

if __name__ == '__main__':
    analyze_travel_budget()
```

8. Test Cases

Test ID	Description	Input Sample Context	Expected Validation Behavior
TC-01	Standard Multi-Currency Mix	EUR 1200, JPY 45000, GBP 150, USD 800	Converts all to USD, applies 2% fee, verifies against $5000.
TC-02	Zero Surcharge Evaluation	USD 1000 with fee_rate modified to 0.00	Ensures conversion vector exactly preserves parity values.
TC-03	Extreme Overflow Trigger	EUR 8000 Luxury Cruise Accommodation	Correctly calculates deficit overruns and flags critical state.

9. Sample Output Screenshots

```
============================================================
ITEM                 AMOUNT     CURR   CONVERTED (USD)
============================================================
Hotel Paris          1200       EUR    $       1346.40
Bullet Train Tokyo   45000      JPY    $        312.12
London Dinner        150        GBP    $        195.84
Flight Transfer      800        USD    $        816.00
```

```
============================================================
Total Consolidated Expenses: $2670.36
Allocated Budget Ceiling:   $5000.00
------------------------------------------------------------
SYSTEM STATUS: SUCCESS [Within Budget Constraints]
============================================================
```

10. Conclusion

This case study effectively demonstrates how multi-currency accounting matrices can be dynamically managed and normalized inside Python. By utilizing systematic lookup maps, looping constructs, and safety conditional splits, the architecture simplifies financial tracking and ensures immediate visibility into budget overruns.

[End of Case Study 9 Documentation]